Open in app ↗

Medium Search Write A

Get unlimited access to the best of Medium for less than $1/week. Become a member

Expedia Group Technology

EXPEDIA GROUP TECHNOLOGY — DATA

Apache Spark Structured Streaming — Checkpoints and Triggers (4 of 6)

Streaming with confidence, resilience, and efficiency



Neeraj Bhadani · Follow

Published in Expedia Group Technology · 6 min read · Feb 25, 2021

196 2



Photo by Alexey Ruban on Unsplash

In previous blog posts, we covered using sources and sinks in Apache Spark™ Streaming. Here we discuss **checkpoints** and **triggers,** important concepts in Spark Streaming.

You may also be interested in some of my other posts on Apache Spark.

- Apache Spark Structured Streaming — Overview
- Apache Spark Structured Streaming — Input Sources
- Apache Spark Structured Streaming — Output Sinks
- Deep Dive into Apache Spark DateTime Functions
- Working with JSON in Apache Spark

- Deep Dive into Apache Spark Window Functions
- Deep Dive into Apache Spark Array Functions
- Start Your Journey with Apache Spark

Setup

Let's start creating a streaming DataFrame (`resultDF`) from a file source by reading a file in each micro-batch and performing aggregations. We use this `resultDF` for all examples in this blog post.

```scala
// Import Libraries
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._
import org.apache.spark.sql.Column
import org.apache.spark.sql.streaming.Trigger
import org.apache.spark.sql.types._

// Create Spark Session
    val spark = SparkSession
      .builder()
      .master("local")
      .appName("Check Point")
      .getOrCreate()

// Set Spark logging level to ERROR.
    spark.sparkContext.setLogLevel("ERROR")

// Define Schema
    val schema = StructType(List(
      StructField("Date", StringType, true),
      StructField("Open", DoubleType, true),
      StructField("High", DoubleType, true),
      StructField("Low", DoubleType, true),
      StructField("Close", DoubleType, true),
      StructField("Adjusted Close", DoubleType, true),
      StructField("Volume", DoubleType, true)
    ))

    // Extract the Name of the stock from the file name.
    def getFileName : Column = {
      val file_name = reverse(split(input_file_name(), "/")).getItem(0)
      split(file_name, "_").getItem(0)
    }

    // Create Streaming DataFrame by reading data from socket.
    val initDF = (spark
      .readStream
      .option("maxFilesPerTrigger", 1)
      .option("header", true)
      .schema(schema)
      .csv("data/stream")
      .withColumn("Name", getFileName)
      )

//     Some aggregations transformation
```

```
// Some aggregations transformation.
val resultDF = initDF.select("Name","Date", "Open", "High", "Low")
  .groupBy(col("Name"), year(col("Date")).as("Year"))
  .agg(avg("High").as("Avg"))
    .withColumn("timestamp", current_timestamp())
```

ss_cp_tr_setup.scala hosted with ❤ by GitHub view raw

Checkpoints

A checkpoint helps build fault-tolerant and resilient Spark applications. In Spark Structured Streaming, it maintains intermediate state on HDFS compatible file systems to recover from failures. To specify the checkpoint in a streaming query, we use the `checkpointLocation` parameter.

Let's understand this with an example. We use the `resultDF` streaming DataFrame defined in the Setup section above.

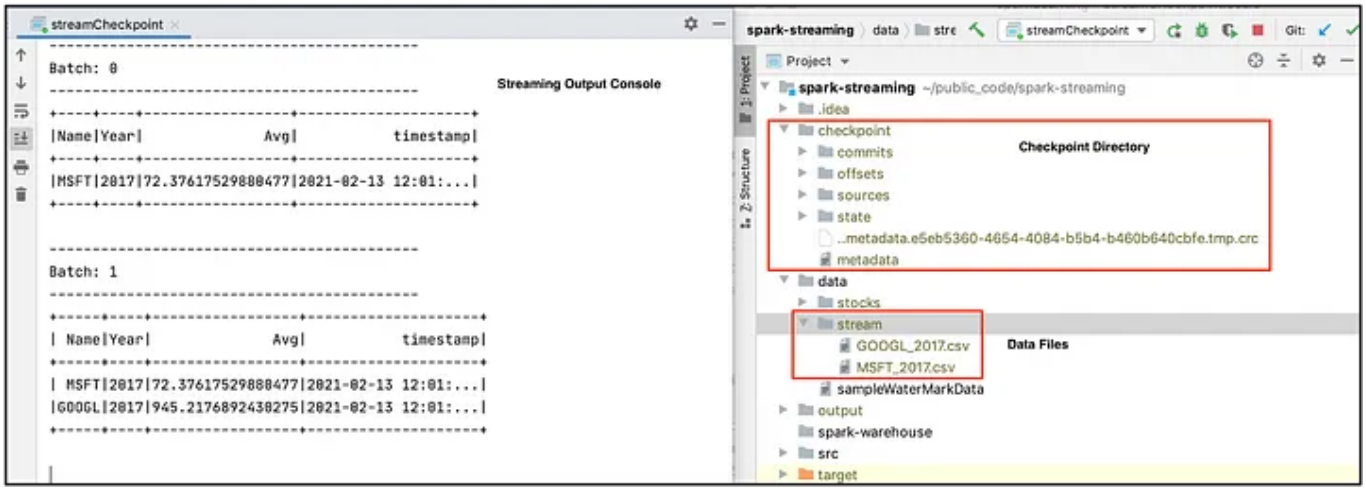
Output to console

```
resultDF
  .writeStream
  .outputMode("complete") // Try with "update" mode as well.
  .option("checkpointLocation", "checkpoint")
  .format("console")
  .start()
  .awaitTermination()
```

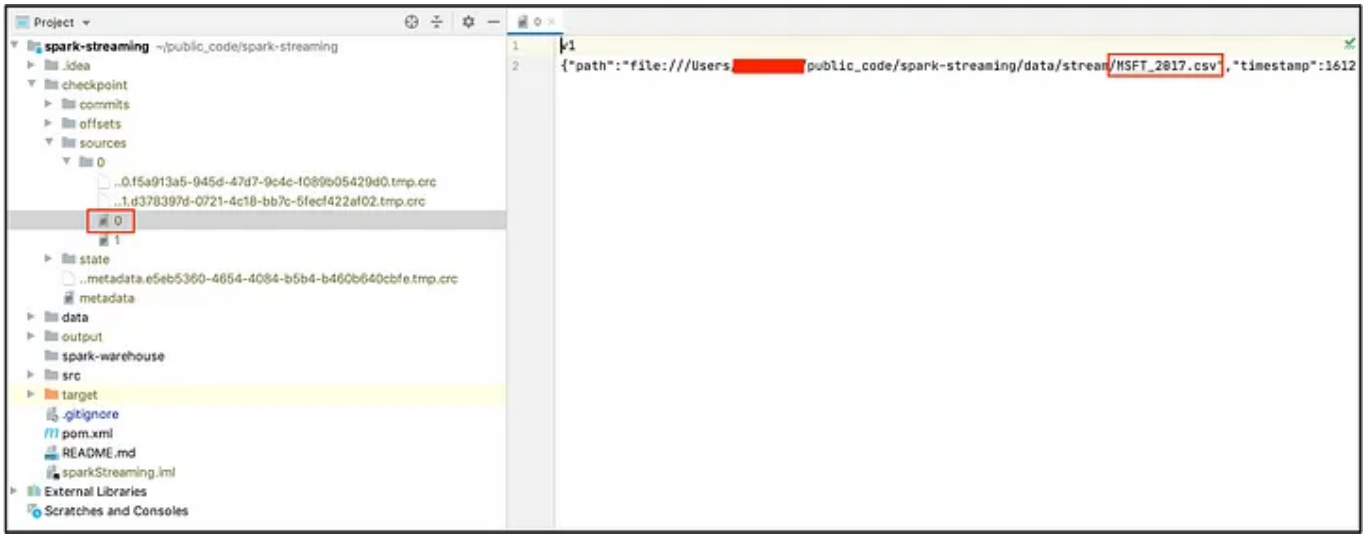
ss_cp_console.output.scala hosted with ❤ by GitHub view raw

The parameter `"checkpointLocation"` enables the checkpoint and specifies the location where we keep checkpoint information.

Let's execute the application and inspect the checkpoint directory.

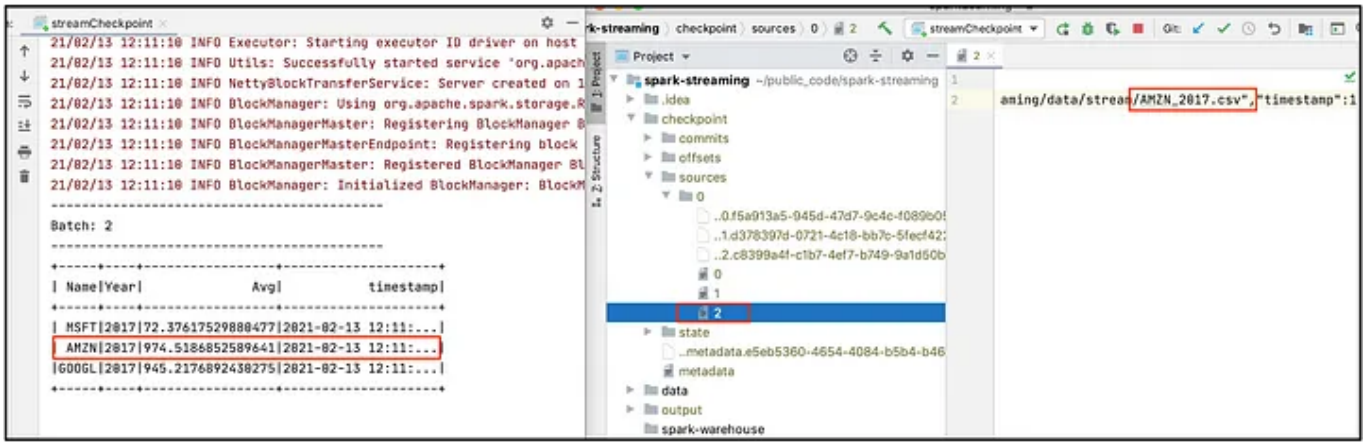


We see a directory named `checkpoint` with a bunch of sub-directories and files. This folder contains the state of our streaming application. E.g. the `sources` folder contains batches of data processed so far.



This screen shot shows the contents of the `sources` folder. In this example, we executed 2 batches with 1 file in each micro-batch. You can see the file `MSFT_2017.csv` was the first to be processed.

Now let's interrupt our streaming application to simulate a failure and start the application again.



Here we see an application started with `Batch: 2` since it already processed `Batch: 0` and `Batch: 1`. We also see another file, named `2`, under the `sources` folder corresponding to `AMZN_2017.csv`. This is how our application recovered from a failure. You can find the complete code on ***GitHub***.

Note: We are getting all records for `MSFT`, `GOOGL` and `AMZN` because we are running in `complete` output mode.

Triggers

By definition, data continuously flows into a streaming system. The arrival of data is not novel enough to kick off processing. In streaming systems, we need a special event to kick off processing and that event is called a trigger. Let's discuss a few triggers in Spark Streaming.

- **Default**: Executes a micro-batch as soon as the previous finishes.
- **Fixed interval micro-batches**: Specifies the interval when the micro-batches will execute. For example, `1 minute`, `30 seconds` or `1 hour` etc.
- **One-time micro-batch**: Executes only one micro-batch to process all available data and then stops.

Default

If we don't specify any trigger, then our query will execute in micro-batch mode. The default trigger executes the next batch as soon as the previous one finishes. In our checkpoint example, we used the default trigger since we hadn't specified another.

Fixed interval micro-batches

Micro-batches are processed after a user-specified time interval. If the processing time of the previous batch is more than the specified interval, the next batch will be executed immediately. If we set `1 minute` as the interval and a micro-batch takes 35 seconds, then the next batch will trigger after waiting for 25 seconds. If a micro-batch takes 70 seconds, then the next micro-batch execute immediately after the first ends.

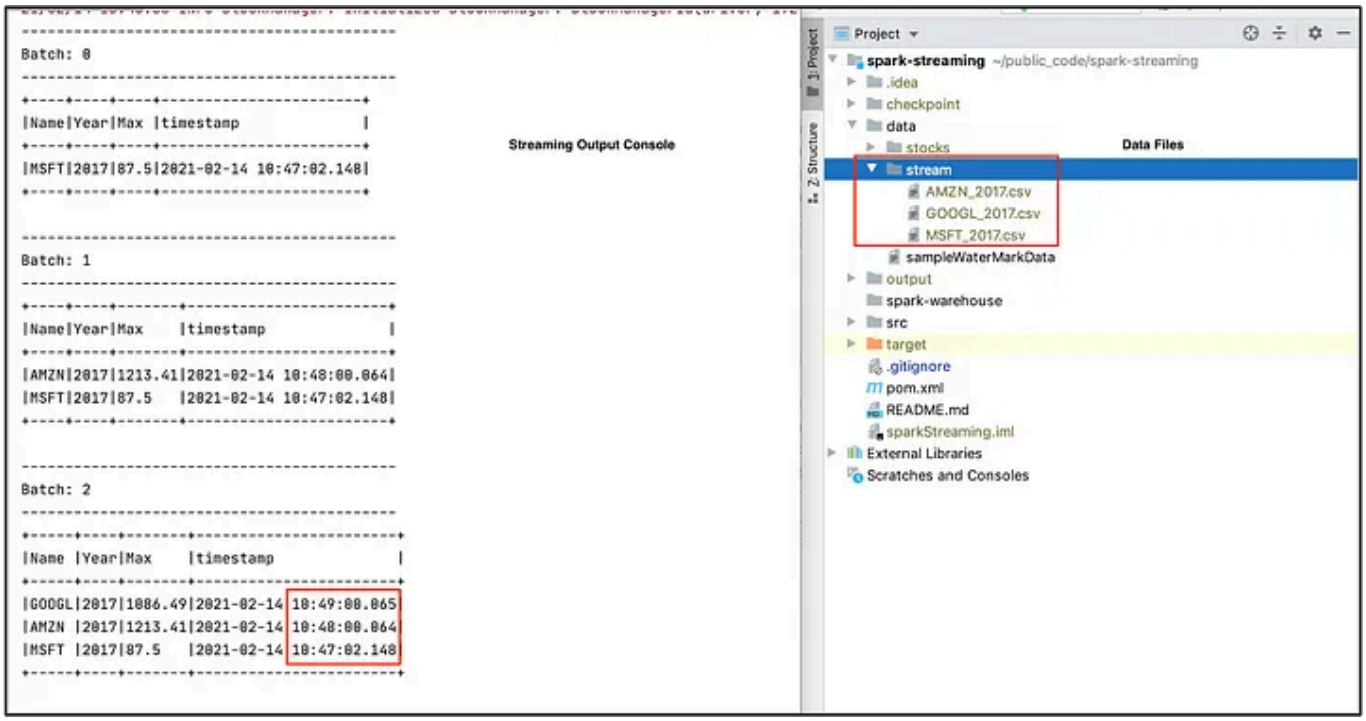
Time for an example. Let's read data from a file source, 1 file in each micro-batch, and print the result to console in `complete` mode. This way we can compare all the processed records. We process micro-batches every `1 minute`. Again, we start with the `resultDF` streaming DataFrame defined above.

Here is a snippet for a fixed interval trigger set at `1 minute`.

```
resultDF
  .writeStream
  .outputMode("complete")
  .trigger(Trigger.ProcessingTime("1 minute"))
  .format("console")
  .option("truncate", false)
  .start()
  .awaitTermination()
```

ss_tr_console_output.scala hosted with ❤ by GitHub view raw

Let's execute our streaming application!



We have 3 files in the `data/stream` folder and specified to process 1 file in each micro-batch. Thus, we see 3 micro-batches in our output. Upon carefully inspecting the timestamp column in our output, we can see `Batch: 0` was executed when we started our application and each subsequent micro-batch was executed at the start of every minute because we specified a `1 minute` interval.

One-time micro-batch

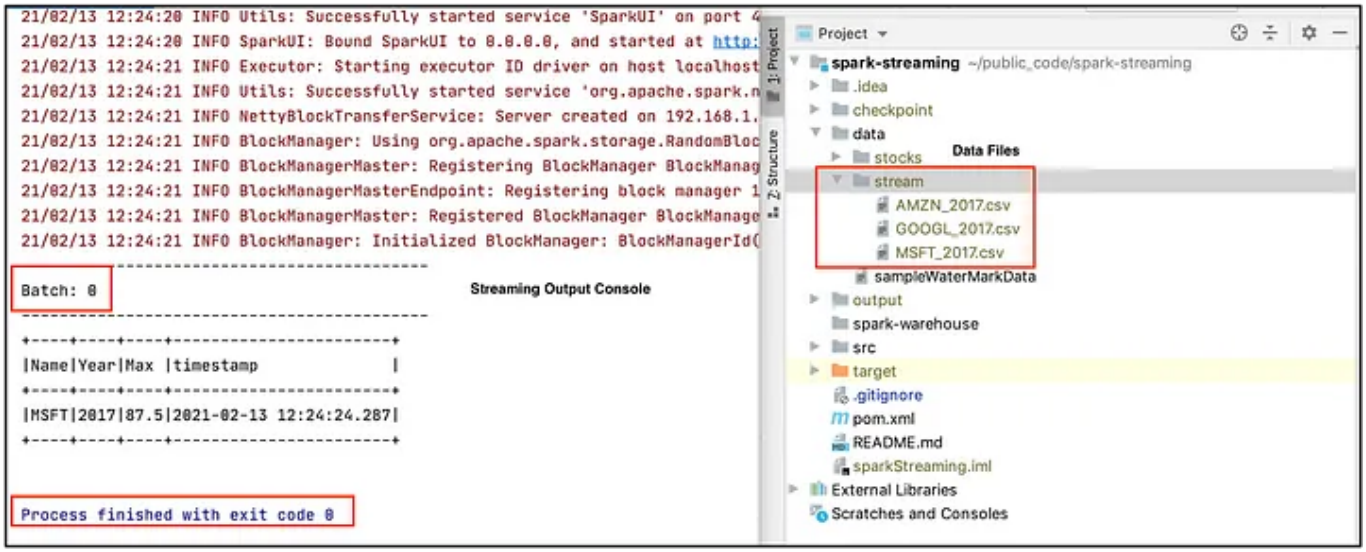
With a once trigger, our query will execute a single micro-batch. It will process all available data and then stop the application. This trigger is useful when you would like to spin-up a cluster periodically, to process all available data, and then shutdown the cluster. This may lead to considerable cost savings.

We reuse the set-up from the fixed interval trigger example, except we set the trigger strategy to once. We also reuse the familiar `resultDF` streaming DataFrame defined above in the Setup section. Here is the snippet for a trigger once.

```scala
resultDF
  .writeStream
  .outputMode("complete")
  .trigger(Trigger.Once())
  .format("console")
  .option("truncate", false)
  .start()
  .awaitTermination()
```

ss_tr_once.scala hosted with ❤ by GitHub view raw

Let's execute our streaming application with the once trigger.



Our streaming application has only executed 1 micro-batch and completed/stopped successfully. Although we have 3 `csv` files in the `data/stream` folder, it has only processed 1 file because we specified only 1 file should be processed in each micro-batch. You can find the complete code on ***GitHub***.

The next post in this series covers Structured Streaming Operations like Filters, Joins, Windows, and User-Defined Functions

Here are other blogs on Apache Spark Structured Streaming series.

- Apache Spark Structured Streaming — First Streaming Example
- Apache Spark Structured Streaming — Input Sources
- Apache Spark Structured Streaming — Output Sinks
- Apache Spark Structured Streaming — Operations
- Apache Spark Structured Streaming — Watermarking



I hope you have enjoyed learning about important concepts like checkpoints and triggers in Spark Streaming.

Reference:

- http://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#triggers
- http://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#recovering-from-failures-with-checkpointing

This blog post is provided for educational purposes only. Certain scenarios may utilize fictitious names and test data for illustrative purposes. The content is intended as a contribution to the technological community at large and may not necessarily reflect any technology currently being implemented or planned for implementation at Expedia Group, Inc.

Software Engineering | Streaming | Technology | Data Science | Spark



Published in Expedia Group Technology

5K Followers · Last published Feb 25, 2025

Follow

Stories from the Expedia Group Technology teams



Written by Neeraj Bhadani

696 Followers · 25 Following

Follow

Data Science and cloud computing enthusiast

Responses (2)

A Akhedkar

What are your thoughts?

P Paul Chung
Apr 13, 2023

I'm curious as to what best practice is for checkpoints after merging changes. Spark seems to typically fail starting again after changes are merged.

Reply

Tim Shen
Sep 24, 2021

In prod environment, when using Kafka input, what should we do if our checkpoint directory gets outdated and the checkpoint log is no longer recognized by Spark? For example, Spark updates a major version

Reply

More from Neeraj Bhadani and Expedia Group Technology



In Expedia Group Technology by Neeraj Bhadani

Deep dive into Apache Spark Window Functions

Window functions operate on groups of data and return values for each record or group

Jun 25, 2020 557 4



In Expedia Group Technolo... by Christine Mathies...

A Short Introduction to Apache Iceberg

The data lake team at Expedia Group starts working with table formats, adds Hive...

Jan 26, 2021 662 3



In Expedia Group Tech... by Avinash Reddy Penu...

How to Import Public Certificates into Java's Truststore from a...

Troubleshooting troublesome certificates

Jul 2, 2020 238



In Expedia Group Technology by Neeraj Bhadani

Deep Dive into Apache Spark Array Functions

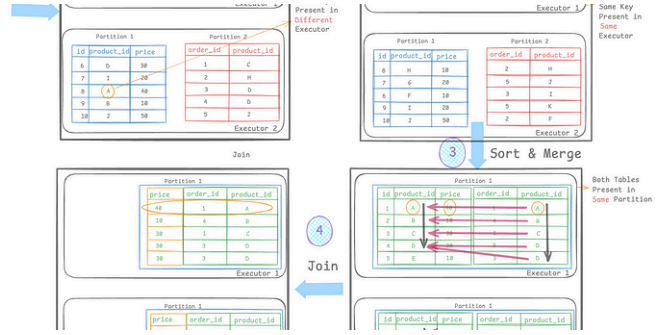
A practical guide to using array functions

Jul 16, 2020 201 3

See all from Neeraj Bhadani

See all from Expedia Group Technology

Recommended from Medium



Shantanu Tripathi



In Data Engineer Things by Ankush Sharma

Sort Merge Join: Visual Flow inside Executors

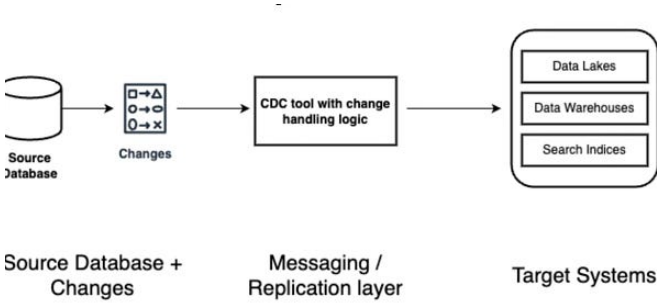
Learn the default join strategy used by Spark through visuals.

Dec 29, 2024 60

The Hidden Optimization Behind Spark's mapPartitions

Iterator-to-Iterator Transformations

Nov 2, 2024 76



Hung Vo

Approaches to Change Data Capture (CDC) in Data Lakes:...

Learn to implement Change Data Capture (CDC) in data lakes using Apache Hudi & AW...

Jan 5 1

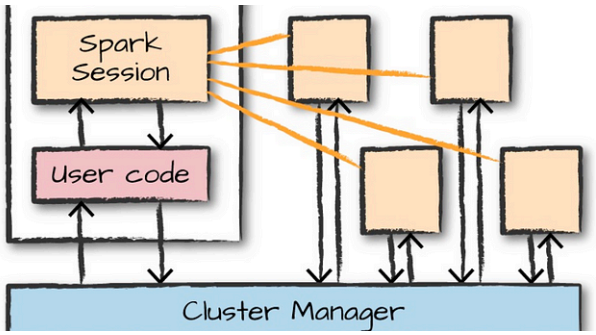


In Dev Genius by Muttineni Sai Rohith

Dynamic Schema Evolution in Pyspark Dataframes

In the world of big data, dealing with changing data structures is a common challenge. Data...

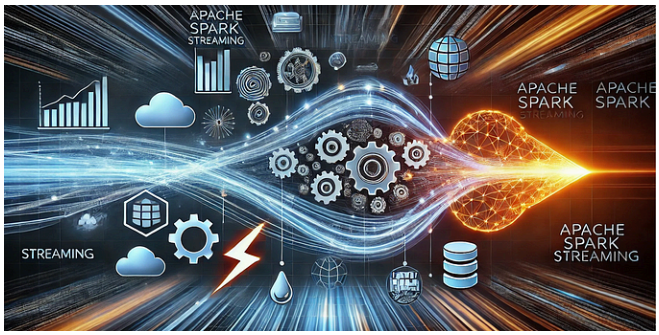
Feb 21 10



In Towards Dev by Prem Vishnoi(cloudvala)

Apache Spark Architecture :A Deep Dive into Big Data Processing

Agenda



Ashutosh Kumar

Apache Spark Streaming — All you need to know...

If you are working in the Analytics, Machine Learning or Data Engineering space, you...

Feb 6 90 1 ••• Jan 2 6 •••

See more recommendations